Code Inspection Report

CollaborAITE

Prepared by The Slice Is Right

COS 397 - Computer Science Capstone 1

March 11th, 2026

Version 1.0.0

Table of Contents

Page

1. Introduction..... 3
   1.1 Purpose of This Document..... 3
   References..... 3
   1.2 Coding and Commenting Conventions.....3
   1.3 Defect Checklist..... 3
2. Code Inspection Process.....4
   2.1 Description.....4
   2.2 Impressions of the Process..... 5
   2.3 Inspection Meetings.....6
3. Modules Inspected.....6
4. Defects..... 10
Appendix A - Coding and Commenting Conventions.....15
Appendix B – Team Review Sign-off.....17
Appendix C – Document Contributions..... 18

1. Introduction

Introduce your project here. Introduce your client, their company or research center and their mission. Introduce your team. Make it complete, but keep it brief.

1.1 Purpose of This Document

The purpose of this document is to describe the code inspection process conducted for the CollaborAITE project developed by The Slice Is Right team. This report documents the team's coding standards and conventions, the checklist used to identify potential defects, and the procedures followed during the inspection process. The goal of the report is to demonstrate that the project's source code has been systematically reviewed to identify errors, enforce consistent coding practices, and improve overall software quality before further development and testing.

References

[1] CollaborAITE System Requirements Specification (SRS), Version 1.0
[2] CollaborAITE System Design Document (SDD), Version 1.0
[3] COS 397 System Design Document Template, Susan Mitchell (Adapted)

1.2 Coding and Commenting Conventions

In this project, we strive for clarity, consistency, and maintainability. Our conventions blend well-established industry standards with a few team-specific adaptations to fit the React/TypeScript + Playwright/e2e testing stack we're working with. We adhere to Airbnb's JavaScript/React style guide as enforced by ESLint/Prettier, follow Microsoft's TypeScript handbook recommendations (explicit exports, readonly, avoid any), and use functional components with hooks in React. Comments are reserved for explaining "why" rather than "what," and public APIs get JSDoc/TSDoc. Unit tests mirror the Arrange-Act-Assert pattern, Playwright specs live alongside code, and we use Conventional Commits so changelogs can be generated automatically.

Most of these rules are adopted verbatim from the linked sources, but we've layered on a handful of project-specific tweaks to keep the codebase navigable and our team productive. For example, we disabled Airbnb's import/no-extraneous-dependencies rule for test files and enforced a use…Context hook-naming convention, and preface each ErrorBoundary component with a comment about expected error types. Appendix A below documents every such home-grown convention with rationale and examples.

1.3 Defect Checklist

- Accessibility

- UI/Layout
- Correctness
- Coding Convention Violation
- User Friendliness
- Correctness/Security
- Coding Convention
- Commenting Convention
- Performance
- Data Integrity
- State Consistency

2. Code Inspection Process

2.1 Description

Our code inspection process began with a preparatory exchange with our sister team, who provided us with an “Intentional Departures” document outlining any deviations between their implemented codebase and the system design document. This pre-inspection artifact was particularly valuable, as it allowed our team to calibrate expectations before diving into the review and avoid spending time flagging known, deliberate divergences. Our sister team also walked us through the general structure of their codebase and the overall data pipeline during an initial joint meeting. From that walkthrough, team members independently identified areas of interest and formulated targeted questions to guide the inspection.

Following the joint meeting, our team divided responsibility for completing the Code Inspection Report (CIR) by assigning different sections to individual members. This division of labor allowed each reviewer to develop deeper familiarity with a specific aspect of the codebase rather than spreading attention too thin across the entire project. Some members concentrated on identifying and categorizing defects, while others focused on understanding architectural patterns and the overall flow of the system. Team members communicated asynchronously throughout this phase, keeping one another informed of findings and ensuring that individually assigned sections remained cohesive.

While our process was functional and ultimately yielded meaningful results, it did diverge somewhat from what might be considered the ideal inspection process. Specifically, we had limited opportunities for collaborative discussion with our sister team beyond the initial walkthrough, and additional joint review sessions could have provided useful clarification on ambiguous sections of the code. Similarly, more structured internal meetings among our own team may have strengthened the consistency of our evaluation criteria across sections. That said, our asynchronous communication approach kept the work moving efficiently given the time constraints,

and we are confident that our process was well-suited to the scope and timeline of this inspection.

2.2 Impressions of the Process

The code inspection process was somewhat effective overall, though its structure as a cross-team review introduced both benefits and limitations. Having our sister team conduct the inspection brought a new and genuine perspective to our codebase – they were able to flag issues we had become too familiar with to notice ourselves, such as the HIPAA violation in the DICOM-to-PNG pipeline leaking patient metadata, the GPU memory leak in the 3D viewer, etc. These are meaningful finds that demonstrate the value of having an outside look on the code. However, one issue that we found is that since the inspectors on the sister team are not deeply familiar with our systems architecture or NFRs, since some findings – like the unconditional 1.2-second timeout in the export module and the session storage bug in settings – are the kind of issues an inspector with more context might be able to probe further into rather than flagging it at surface level.

If we were to approach the process differently next time, It would be nice to have a member of the sister team be more briefed and caught up with the process of the codebase similar to our client liaison role. A deeper understanding of our NFR documentation briefing would also aid in helping the inspectors understand why and how our product does what it does and what our plans are. A shared understanding between the two teams in advance of the CIR would have made the findings more meaningful and actionable. We do understand that since the class has structured time something like this suggestion could actually be represented in a real life scenario but not in our class time.

The 3D viewer and Upload page are our strongest modular units in terms of likelihood of remaining flaws. The viewers memory leak was a specific report from our sister team allowing us to better understand and delve into the problem at hand for it, beyond this the inspection gave us reasonable confidence in our overall structure and systems. The upload page similarly received adequate contained feedback. The settings page, on the other hand, is the weakest module – the session storage bug represents a fundamental persistence failure overlooked due to us not using cookies to mitigate HIPAA concerns. Despite being our weakest module the settings page was also one that we had the least amount of requirements for, meaning that we effectively put the least amount of effort into it.

2.3 Inspection Meetings

Our team held one formal code inspection meeting with our sister team on March 4, 2026, conducted in person during class time. The meeting took place on the University of Maine campus and ran from 2:00 pm to 3:15 pm.

Participants and Roles:

Role	Member	Responsibilities
Reader	Israk	Led the sister team through our team's general code structure and overall application pipeline
Moderator	Greg	Kept the meeting on task, ensured all participants remained aligned and managed the flow of the discussion
Scribes	Cooper, Ethan	Both took comprehensive notes throughout the meeting to support post-meeting CIR completion
Inspector	Bryan	Formulated and posed questions to the sister team as they walked through their codebase

Given the breadth and complexity of the sister team's codebase, the meeting did not drill down into specific code units in detail. Instead, the session prioritized orienting our team to the high-level architecture and application pipeline. This was a deliberate choice that proved more practical given the size of their project. The sister team guided us through their module organization and the general flow of data and control through the application, equipping our team to conduct more focused independent review of individual modules following the meeting. Specific code units were then examined during individual inspection work completed after this session.

3. Modules Inspected

Our inspection covered the major frontend and backend modules that implement group collaboration, messaging, AI interaction, and journal integration. Based on the current state of the repository, the codebase appears to be more than 75% complete. Core features such as authentication, group management, messaging, AI policy enforcement, journaling, and real-time updates are implemented. The most significant architectural extension beyond the SDD is the addition of conversation-based chat channels within groups. The SDD describes a simpler group-to-messages structure, while the implemented system supports general, group, and private conversations inside each group.

The following modules were inspected because they represent the core user-facing collaboration workflow and the backend services that support it.

1. Module: ChatPage.tsx and ChatInterface.tsx

Functionality: These frontend modules provide the main entry point for group chat. ChatPage loads the selected group and renders the chat area. ChatInterface coordinates message display, message submission, typing indicators, helpful-vote actions, message editing and deletion, and AI-related policy checks before sending prompts.
Fit in SDD and Architecture: These modules correspond to the Web Client layer in the SDD. They implement the user interface for collaboration and chat and connect the React frontend to backend APIs and real-time listeners.
Planned vs. Actual Design: The SDD describes group chat as a direct interaction with group messages. The actual implementation supports that baseline but also includes additional logic for AI policy checks, typing indicators, optimistic interactions, and toast-based error feedback. In interface terms, the implementation is broader than the SDD because it accepts optional members, highlight targets, journal-linking callbacks, and role-aware behavior. This increases usability and feature coverage, but it also increases complexity and introduces more UI state transitions that must be tested.

2. Module: ConversationView.tsx, ConversationList.tsx, ConversationHeaderMenu.tsx, and ConversationSettingsModal.tsx

Functionality: These modules implement a conversation system inside a group. ConversationList renders available channels, including the pinned General conversation and other group or private conversations. ConversationView displays the selected conversation, its messages, typing indicators, journal integration, and conversation management controls. ConversationHeaderMenu and ConversationSettingsModal provide update and deletion actions.
Fit in SDD and Architecture: These modules still fit the frontend chat portion of the SDD, but they extend it substantially. The SDD assumes a flatter group chat model. The implemented code introduces an intermediate conversation abstraction that is not explicitly present in the original design.
Planned vs. Actual Design: The planned interface in the SDD centers on groups and group messages. The actual interface adds conversations with types such as general, group, and private. The planned data structure centers on groups/{groupId}/messages. The actual code also uses conversation-specific message paths and conversation metadata such as title, description, participants, and updated timestamps. The tradeoff is clear: the implemented design is more reusable and better supports scoped discussion and private collaboration, but it introduces additional routing, authorization rules, and data relationships that are more complex than the original design.

3. Module: chat.service.ts and chat.controller.ts

Functionality: These backend modules implement the legacy group-message path. They support sending, retrieving, searching, editing, deleting, and resolving messages stored directly under a group. They also support helpful-vote toggling for AI-generated messages.
Fit in SDD and Architecture: These modules align closely with the SDD's backend layer and Firestore schema. The SDD explicitly defines group messages under groups/{groupId}/messages, and this service implements that path directly.
Planned vs. Actual Design: The planned interface is mostly preserved. The actual implementation extends it with edit, delete, search, resolve, and helpful-vote features, which improve usability and journal integration. The actual data structures also include metadata such as mentions, edited state, and AI-related flags. This is a reasonable extension of the SDD because it preserves the original storage path while supporting later use cases.

4. Module: conversation.service.ts, conversation.controller.ts, and conversation.routes.ts

Functionality: These backend modules implement conversation CRUD operations and conversation-scoped messages inside groups. They ensure a General conversation exists, create additional group or private conversations, enforce visibility rules, manage participants for private conversations, and store messages in nested subcollections.
Fit in SDD and Architecture: This module cluster is the clearest deviation from the original SDD. It still belongs within the backend chat subsystem, but it adds a new structural layer between groups and messages.
Planned vs. Actual Design: The planned interface did not include conversations as a first-class resource. The actual implementation adds routes under group conversation endpoints and stores messages under groups/{groupId}/conversations/{conversationId}/messages. The planned data structures were simpler; the actual structures include conversation documents with creator, participants, message count, last-message preview, provider selection, and timestamps. The main tradeoff is that the conversation layer improves organization and supports private or AI-focused discussion, but it also increases coupling between frontend state, route design, and access-control logic.

5. Module: ai-policy service cluster, including ai-policy.service.ts, policyEvaluator.service.ts, llm.service.ts, pseudonymizer.service.ts, and depseudonymizer.service.ts

Functionality: These modules enforce AI usage rules, validate prompts against instructor policy, pseudonymize sensitive data before external model calls, dispatch

requests to supported providers, and translate responses back into the application's context.
Fit in SDD and Architecture: This cluster fits directly into the AI Gateway and Policy Enforcement layer defined in the SDD. It is one of the strongest matches between design and implementation.
Planned vs. Actual Design: The SDD calls for prompt validation, pseudonymization, provider routing, and response filtering. The code implements those responsibilities and goes further by adding caching, multi-provider support, context-window management, and reversible pseudonym maps. The implemented algorithms are therefore more mature than the baseline design. The tradeoff is increased sophistication and better compliance support in exchange for more moving parts and more failure modes that must be covered by tests.

6. Module: journals.service.ts, journals.controller.ts, journals.routes.ts, and the related journal UI components

Functionality: These modules implement journal creation, retrieval, sharing, and linking journal entries to chat messages. The journal UI also supports linked-message display and quick journal creation from chat context.
Fit in SDD and Architecture: These modules align with the journaling functionality described in the SDD and with the platform's reflective-learning mission.
Planned vs. Actual Design: The SDD describes journal entries as separate records with group linkage and visibility settings. The implementation follows that model and adds direct message-linking behavior through linked message identifiers. This improves traceability between discussion and reflection. The tradeoff is that journal features become dependent on stable message-resolution behavior across both the legacy flat message path and the newer conversation-based path.

7. Module: groups.routes.ts and related group services

Functionality: These modules act as the integration point for group membership, invites, nested messages, policies, analytics, and conversations.
Fit in SDD and Architecture: This module fits squarely in the backend group-management portion of the SDD and serves as a coordination layer for nested resources.
Planned vs. Actual Design: The SDD expects group-centered APIs, and the implementation preserves that structure. The actual code extends the nested resources considerably by mounting conversation, analytics, policy, and prompt-configuration routes under groups. This is consistent with the overall architecture even though the breadth of nested functionality is greater than the SDD originally showed.

Incomplete or Partially Completed Modules:

The repository appears substantially complete, but several modules or subsystems remain incomplete or partially implemented.

1. Module: Matching and Scheduling Service

Description: The SDD describes a specialized service for pairing users and coordinating scheduled sessions, including Google Calendar integration. This subsystem does not appear to be implemented in the inspected codebase.

2. Module: Production Email Delivery Service

Description: Email invitation support exists, but the current backend email service is a console-based stub for the prototype rather than a real delivery provider.

3. Module: Google Calendar Integration

Description: The SDD includes calendar integration as part of the broader system architecture, but no concrete implementation was found in the inspected modules.

4. Module: Placeholder Frontend Pages

Description: A small number of frontend routes still contain placeholder content, including settings and account-related pages. These do not block the core collaboration workflow but indicate that some UI work remains.

Overall Assessment

Overall, the project has progressed well beyond the minimum threshold expected for code inspection. The strongest implementation areas are authentication, groups, chat, AI policy enforcement, and journaling. The most important difference between the SDD and the code is the addition of the conversation layer inside groups. This extension improves modularity, supports private and scoped discussions, and better fits the platform's collaborative AI goals. However, it also creates a meaningful divergence from the original design and should be documented clearly in the report as an implementation extension rather than a direct one-to-one realization of the SDD.

4. Defects

Frontend Defects:

Module	Defect Description	Category
AIErrorMessage.tsx	Outer is missing role='alert', making dynamically	Accessibility

	rendered error messages invisible to screen readers and assistive technologies	
AIErrorMessage.tsx	The <p> rendering errorMessage lacks an overflow-wrap/break-words class, causing long unbroken strings to overflow the container and break the layout	UI/Layout
AIMessage.tsx	prevHelpfulRef is assigned helpfulVotes on the same render, making prevHelpfulRef !== to helpfulVotes always false	Correctness
AIMessage.tsx	setOptimisticVotes(null) is called directly in the render body, violating Reacts rules. Triggering a state update mid-render can cause cascading re-renders and visual glitches.	Coding convention violation
AIOptionsBar.tsx	A is nested inside a <button>, which is invalid HTML	Accessibility
AIOptionsBar.tsx	Decorative icons inside toggle buttons are missing aria-hiddin="true". Could cause issues for screen readers	Accessibility
ChatInterface.tsx	toastError uses 'in' operator without guarding against null/undefined. Throws a TypeError instead of falling back gracefully	Correctness
ChatInterface.tsx	handleSend shows a toast error and then re-throws, causing the user to see duplicate error feedback for the same failure	User Friendliness
ChatInterface.tsx	handleHelpfulToggle has no try/catch and provides no user feedback on failure	User Friendliness
ConversationHeaderMenu.tsx	All four buttons are missing type="button". Clicking the menu item submits the form instead of executing its handler.	Correctness
ConversationHeaderMenu.tsx	Trigger button is missing aria-expanded and aria-haspopup="menu", meaning screen reader users will have no way to know the button controls a popup menu or whether it is currently open or closed	Accessibility
ConversationHeaderMenu.tsx	The backdrop is exposed to the accessibility tree as an anonymous interactive region. It should have aria-hidden="true" so assistive technology ignores it	Accessibility
ConversationList.tsx	All interactive buttons - New Channel, General Conversation, and all buttons in grouped.group.map are missing type="button", defaulting to type="submit" and risking accidental form submission	Correctness

ConversationSettingsModal.tsx	onSave is called without await inside handleSubmit. The modal closes before knowing whether the save succeeded, leaving the user with no feedback if the API call fails	User Friendliness, Correctness
ConversationView.tsx	useToastContext() is destructured as const {toast} = useToastContext(), which is inconsistent with its usage in ChatInterface.tsx. If the context has no nested toast property, every toast.error() call throws a TypeError at runtime	Correctness
ConversationView.tsx	handleHelpfulToggle has no try/catch and provides no user feedback on failure.	User Friendliness
ConversationView.tsx	onSave and onConfirm callbacks passed to modals are fire-and-forget with no await or error handling. If the underlying API call fails, the modal closes, the UI shows stale data, and the user receives no feedback	Correctness
ConversationView.tsx	The “Manage Participants” and “Toggle journal panel” buttons are missing type=”button”, defaulting to type=”submit” and risking accidental form submission	Correctness

Backend Defects:

Module	**Defect Description**	**Category**

app.ts	**cors({ origin: true })** allows requests from any origin. In production this is a serious security vulnerability — it should whitelist specific allowed origins. Not a huge concern considering the app isn't shipped.	Correctness/ Security
app.ts	The 404 handler is missing the next parameter signature **(req, res, next).** While it works, it's inconsistent with Express convention and could cause linting/type errors in strict TypeScript setups.	Coding Convention
app.ts	The health check endpoint at **/health** sits outside the **/api** prefix, which is inconsistent with all other routes. It should either be **/api/health** or this inconsistency should be documented.	Coding Convention
auth.middleware.ts	**requireRole** is marked **@deprecated** but is still exported and presumably still called somewhere. Deprecated functions that do nothing but pass through are a security risk. It should either be removed, throw an error, or log a loud warning on invocation.	Correctness/ Security
auth.middleware.ts	**authMiddleware** is exported as an alias for **authenticateUser** for "backward compatibility" but there's no documentation of what it's backward-compatible with, when the alias was introduced, or when it's safe to remove.	Commenting Convention
rate-limit.middleware.ts	The in-memory store maps never evict expired entries. A long-lived instance accumulates one entry per unique IP/user indefinitely. Under sustained traffic this is an unbounded memory leak.	Correctness
aiGateway.controller.ts	require('../../shared/socket/socketServer') executes inside an LLM streaming callback that fires for every chunk (~500+ times per response). Dynamic requires are expensive and should load once before streaming starts, not repeatedly in the loop.	Performance
policies.service.ts	When updating a policy, if existingData.createdAt can't convert to .toDate(), the code falls back to new Date(), creating a brand new timestamp instead of preserving the original creation time. This destroys audit trails and violates immutability of metadata fields.	Data Integrity
promptConfig.service.ts	Method sends FieldValue.serverTimestamp() to Firestore but returns new Date() to the client. Clock skew means returned value won't match the persisted value, violating single source of truth and causing cache inconsistency bugs.	State Consistency

Commenting Inconsistencies:

Our inspection revealed inconsistent commenting practices across the codebase. While certain files included detailed, well-structured comments that meaningfully aided comprehension, a number of other files contained little to no documentation, or comments too vague to provide actionable context. This inconsistency was most noticeable in files where the underlying logic was non-trivial, as the absence of adequate comments in those areas placed a greater burden on the reader to infer intent from the code alone. Establishing and enforcing a consistent commenting standard across all modules would improve maintainability and reduce onboarding friction for future contributors.

Appendix A - Coding and Commenting Conventions

This appendix outlines conventions adopted specifically for this project that extend or modify standard development practices. These rules were implemented to improve code readability, maintainability, and consistency within the React/TypeScript and Playwright testing environment.

A.1 Dev-Dependencies in Tests

Rationale: Playwright test specifications and helper files are stored alongside the application code they test. Requiring these imports to be listed strictly as devDependencies created unnecessary warnings and added noise during development.

Rule: The ESLint configuration disables import/no-extraneous-dependencies for files located in:

- tests/**
- webapp/**/__tests__/**

A.2 Custom Hook Naming for Context

Rationale: Using a consistent naming pattern allows developers to quickly locate related context hooks, providers, and types using search tools.

Rule: Hooks that return context values must:

- Start with use
- End with the name of the context

A.3 Error Boundary Comments

Rationale: Code reviewers frequently needed clarification on whether an ErrorBoundary component was intended to handle network errors or rendering failures.

Rule: Each ErrorBoundary component file must begin with a one-sentence comment describing the expected source of errors.

A.4 SVG and Asset Import Paths

Rationale: Vite's path aliasing sometimes created confusion for new contributors when importing assets.

Rule: Assets must always be imported using the @/assets/... alias. A corresponding path entry must exist in tsconfig.json.

A.5 Test Fixture Naming

Rationale: The project contains a large directory of DICOM test fixtures. A consistent naming scheme prevents filename collisions and ensures easy identification.

Rule: Fixture filenames use zero-padded integers derived from the SOPInstanceUID.

Appendix B – Team Review Sign-off

This document confirms that all undersigned members of The Slice Is Right project team have thoroughly reviewed the entirety of this Code Inspection Report for the CollaborAITE system. By signing below, each team member acknowledges their understanding of the proposed system architecture and detailed design and formally agrees that the content, scope, and technical direction outlined in this document are accurate, feasible, and provide a suitable foundation for the implementation phase of the project.

The comment section provided for each team member is to be used for noting any minor suggestions, editorial feedback, or non-substantive points of clarification. It is recognized that for this sign-off to be granted, there are no major points of contention regarding the architectural or detailed design decisions outlined within this CIR.

At this time, digital signage is not working with Adobe, we will have this document digitally signed at a later date.

Israk A.	Israk Arafat	3/25/26
Greg M.		3/25/26
Cooper S.	Cooper Stepankiw	3/25/26
Bryan S.	Bryan Sturdivant	3/25/26
Ethan W.	Ethan Wyman	3/25/26

Comments:

Appendix C – Document Contributions

This appendix details the contributions of each team member to the creation of this Code Inspection Review. All members participated in the collaborative design process to ensure a comprehensive and technically sound document. The percentage contributions are estimates that reflect the primary authorship and development effort for the various sections and diagrams.

Member Name	Primary Responsibilities	Estimated Percentage
Israk Arafat	Section 3	25%
Gregory Michaud	Section 2 and 4	20%
Cooper Stepankiw	Section 3 and Appendices	10%
Bryan Sturdivant	Section 1, 2, and 4	25%
Ethan Wyman	Section 2 and 4	20%

Member Name	Primary Responsibilities	Estimated Percentage
Total		100%